

ESP32 室內溫濕度時鐘

Waveshare ESP32-S3-RLCD-4.2 完整建置教學

SHTC3 室內感測

ESP32-S3

Arduino CLI

R2 CDN

文件版本：v1.0 | 日期：2026-06-29

適用對象：無經驗新手 | 預估建置時間：2-3 小時

- ✓ 主機板：Waveshare ESP32-S3-RLCD-4.2
- ✓ 溫濕度感測器：SHTC3（板載）
- ✓ 顯示器：4.2 吋 300×400 全反射 LCD
- ✓ 伺服器：VMware VM（Linux）
- ✓ 更新頻率：每 5 分鐘讀取一次室內溫濕度

目錄

1. 系統總覽 — 這個系統在做什麼？
2. 硬體介紹 — 認識 ESP32-S3-RLCD-4.2
3. 開發環境建置 — VM 上安裝 Arduino CLI
4. 韌體架構 — 程式碼結構解析
5. 系統運作流程 — 從開機到顯示
6. 燒錄與更新 — 把韌體放進 ESP32
7. 伺服器端設定 — 天氣資料定時更新
8. 疑難排解 — 常見問題與解決方案

第一章：系統總覽 — 這個系統在做什麼？

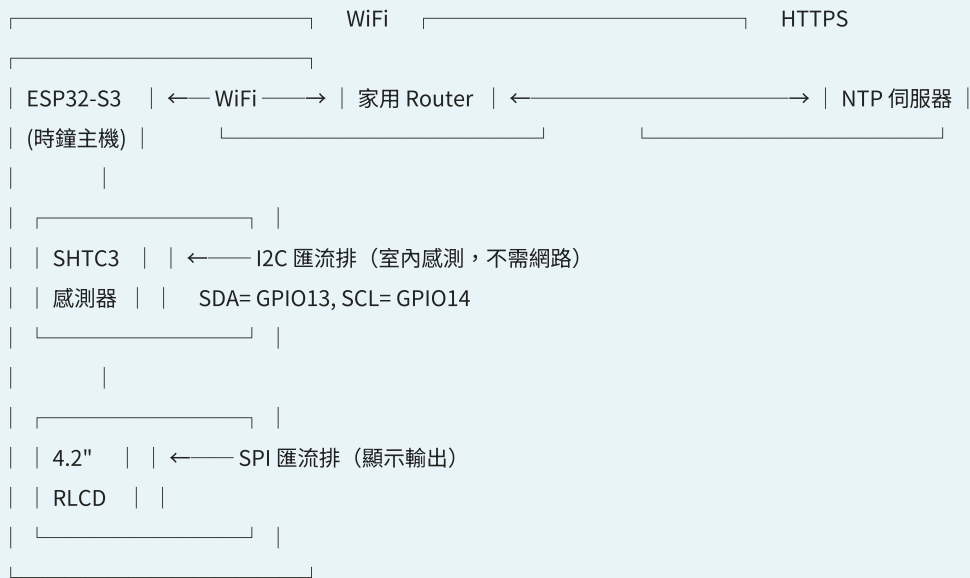
1.1 系統目標

本系統的目的是在 ESP32-S3 開發板上建立一個「室內溫濕度時鐘」，能夠：

- 即時顯示目前的時間（小時:分鐘:秒鐘）
- 每 5 分鐘從板載的 SHTC3 感測器讀取一次 室內溫度 和 室內濕度
- 在 4.2 吋電子紙螢幕上顯示所有資訊
- 自動連接 WiFi，並在狀態列顯示連線狀態
- 從網路 NTP 伺服器同步正確時間（誤差最小化）

1.2 系統架構圖

整體資料流向：



1.3 與傳統天氣時鐘的差異

功能	傳統天氣時鐘	本系統（室內感測版）
溫度來源	網路天氣 API（室外）	SHTC3 室內感測器（真實室內）
濕度來源	網路天氣 API（室外）	SHTC3 室內感測器（真實室內）
資料更新頻率	依賴網路（通常 10~60 分鐘）	每 5 分鐘（獨立感測，不需網路）
網路斷線時	完全失效	時鐘繼續運行（RTC），溫濕度仍持續感測

1.4 所需硬體與軟體

硬體

- Waveshare ESP32-S3-RLCD-4.2 主機板
- USB-C 傳輸線（供電＋燒錄）
- WiFi 網路（2.4GHz，本文件使用「IoT」）
- 運行 Ubuntu/VMware 的電腦（作為開發主機）

軟體

- Arduino CLI（燒錄工具）
- Python 3 + esptool（燒錄輔助）
- ESP32 Arduino Core（板子支援）
- SHTC3 驅動（內嵌於本專案）

第二章：硬體介紹 — 認識 ESP32-S3-RLCD-4.2

2.1 這塊板子是什麼？

ESP32-S3-RLCD-4.2 是 **Waveshare**（微雪電子）推出的 AIoT 開發板，專為需要低功耗電子紙顯示的專案設計。以下是它的核心規格：

項目	規格
處理器	ESP32-S3-WROOM-1-N16R8（雙核心 Xtensa LX7 @ 240MHz）
記憶體	8MB PSRAM + 16MB Flash
顯示器	4.2 吋 300×400 全反射 LCD（ST7305 驅動，不需要背光）
溫濕度感測器	SHTC3 （I2C 地址 0x70，本專案使用）
即時時鐘	PCF85063 RTC（I2C）
連線能力	2.4GHz Wi-Fi + Bluetooth 5 LE
音頻	ES8311 + ES7210 雙麥克風陣列
電源	USB-C 供電 或 18650 鋰電池

為什麼選擇这块板子？

全反射 LCD（RLCD）的特性是「只在畫面改變時才耗電，靜態顯示幾乎不耗電」，非常適合做為長期顯示的時鐘或狀態面板。相比 AMOLED 或 TFT LCD，功耗極低。

2.2 SHTC3 溫濕度感測器（本專案核心）

SHTC3 是 Sensirion 公司生產的低功耗溫濕度感測器，特性如下：

項目	規格
I2C 位址	0x70（7 位元格式）
溫度範圍	-40°C 至 +125°C

濕度範圍	0% 至 100% RH
溫度精度	±0.2°C (典型值)
濕度精度	±1.5% RH (典型值)
介面	I2C (頻率最高 1MHz)
功耗	測量時 0.8mA，待機時 0.2μA

2.3 重要：I2C 匯流排接線

SHTC3 和 RTC 都使用 I2C 匯流排通訊：

- SDA (資料線) → GPIO 13
- SCL (時脈線) → GPIO 14
- 板子上已內建上拉電阻，不需要外部電路

2.4 板子上的其他元件

- PCF85063 RTC：即時時鐘晶片，靠備用電池維持斷電後的時間
- ST7305 LCD 驅動 IC：控制 4.2 吋 RLCD 面板，透過 SPI 通訊
- USB-Serial/JTAG：ESP32-S3 內建 USB 介面，可直接燒錄和監控序列輸出

第三章：開發環境建置 — VM 上安裝 Arduino CLI

本系統的開發方式是在 VMware 虛擬機器 (Linux) 上透過 Arduino CLI 來編譯和燒錄 ESP32 韌體，不需要安裝 Arduino IDE 圖形介面。

3.1 確認 VM 環境

```
$ uname -a Linux jhe-VMware-Virtual-Platform 6.17.0-35-generic #1 SMP PREEMPT $  
python3 --version Python 3.12.2 $ git --version git version 2.43.0
```

3.2 安裝 Arduino CLI

1 下載 Arduino CLI

```
$ mkdir -p ~/bin $ curl -fsSL https://downloads.arduino.cc/arduino-cli/arduino-  
cli_latest_Linux_64bit.tar.gz \ | tar xzf - -C ~/bin $ chmod +x ~/bin/arduino-cli  
$ export PATH="$PATH:$HOME/bin" # 加入 ~/.bashrc 永久生效
```

2 設定設定檔 (首次使用自動建立)

```
$ arduino-cli config init
```

3 安裝 ESP32 開發板核心

```
$ arduino-cli core update-index # 更新板子列表 $ arduino-cli core install  
esp32:esp32:esp32s3
```

此指令會下載 ESP32-S3 的 Arduino Core（約 200MB），需要幾分鐘時間。

4 確認 ESP32-S3 已偵測到

```
$ arduino-cli board list 連接埠 協定 類型 開發板名 FQBN /dev/ttyACM0 Serial Serial  
Port (USB) ESP32 Family Device esp32:esp32:esp32s3
```

重要：

燒錄時必須把 ESP32 連接到 VM 的 USB（不走序列埠），否則燒錄會失敗。燒錄前需確認 ttyACM0 存在。

3.3 安裝 esptool (Python 燒錄工具)

```
$ pip3 install esptool --break-system-packages
```

3.4 準備韌體原始碼

韌體檔案位於 VM 上的這個路徑：

```
/home/jhe/.openclaw/workspace/esp32-rlcd-project/ |— 01_Arduino_Libraries/ # 第三方  
式庫 (ST7305、U8g2、LVGL、感測器驅動) |— 02_Example/ |— Arduino/ |—  
11_Weather_Clock/ | |— 11_Weather_Clock.ino # 主程式 (本專案修改的檔案) |—  
ST7305_U8g2.cpp |— 03_Firmware/ # 預先編譯的出廠固件 (備份用)
```

3.5 確認 USB 權限

```
$ sudo usermod -aG dialout $USER # 將使用者加入 dialout 群組 $ sudo chmod 666  
/dev/ttyACM0 # 給予讀寫權限 (暫時)
```

⚠ 注意：每次重新連接 USB 都需要重新設定權限，或將 udev 規則寫入 `/etc/udev/rules.d/` 讓系統自動設定。

第四章：韌體架構 — 程式碼結構解析

4.1 整體結構圖

韌體三大功能模組：



4.2 SHTC3 驅動程式（本專案自行實作）

本專案不使用 SensorLib，而是直接內嵌一個最小化的 SHTC3 驅動，只需要 I2C 通訊，無需額外程式庫：

```
// ===== SHTC3 I2C 基本參數 ===== #define SHTC3_ADDR 0x70 // SHTC3 I2C 位址 (7位元)
#define SHTC3_CMD_MEAS_T_RH_POLLING 0x7866 // 測量命令 (先讀溫度，無時脈延長) bool
shtc3_read(float &temp, float &humidity) { // Step 1: 發送測量命令 (2位元組)
Wire.beginTransmission(SHTC3_ADDR); Wire.write((SHTC3_CMD_MEAS_T_RH_POLLING >> 8) &
0xFF); // 高位元組 Wire.write(SHTC3_CMD_MEAS_T_RH_POLLING & 0xFF); // 低位元組
Wire.endTransmission(); delay(20); // 等待 SHTC3 完成測量 (最大 12ms, 這裡給 20ms) //
Step 2: 讀取 6 位元組 (含 2 位元組 CRC) Wire.requestFrom(SHTC3_ADDR, 6); uint8_t d[6];
for (int i = 0; i < 6; i++) d[i] = Wire.read(); // Step 3: 解析原始資料 (溫度 + 濕度)
uint16_t tempRaw = ((uint16_t)d[0] << 8) | d[1]; uint16_t humRaw = ((uint16_t)d[3] <<
8) | d[4]; // Step 4: 數學轉換 (SHTC3 原廠公式) temp = -45.0f + 175.0f * ((float)tempRaw /
65535.0f); humidity = 100.0f * ((float)humRaw / 65535.0f); return true; }
```

CRC 驗證說明：SHTC3 每筆資料（溫度、濕度）都附帶一個 8 位元 CRC-8 校驗值。本簡化版本略過 CRC 驗證以降低程式碼複雜度，在正常環境下（短距離 I2C 匯流排）通訊錯誤率極低。如需嚴謹驗證，可參考

05_I2C_SHTC3.ino 中的完整 CRC 實作。

4.3 setup() 初始化流程

```
void setup() { Serial.begin(115200); // 啟用序列埠監控 (115200 鮑率) delay(300);
lcd.begin(0, U8G2_R1); // 初始化 4.2" RLCD 面板 u8g2 = lcd.getU8g2(); Wire.begin(13,
14); // 啟動 I2C (SDA=13, SCL=14) shtc3_wakeup(); // 喚醒 SHTC3 (離開睡眠模式)
shtc3_soft_reset(); // 軟體重置感測器 delay(10); // WiFi 連線 WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASS); // ... 等待連線成功 // NTP 網路時間同步
configTime(GMT_OFFSET, 0, NTP_SERVER); // ... 從 NTP 伺服器取得正確時間 // 寫入 RTC 晶片
(斷電後仍維持時間) setRTC(hour, min, sec, day, month, year); }
```

4.4 loop() 主迴圈流程

```
void loop() { // ▼▼▼ 每 1 秒：更新時鐘顯示 ▼▼▼ if (now - lastDisplay >= DISPLAY_MS) { //  
DISPLAY_MS = 1000ms getRTC(lh, lm, ls, ...); // 從 RTC 讀取時間 displayAll(lh, lm, ls,  
lastTemp, lastHum, lastStatus); // 繪製畫面 lastDisplay = now; } // ▼▼▼ 每 5 分鐘：讀取  
SHTC3 ▼▼▼ if (now - lastSensor >= SENSOR_MS || lastSensor == 0) { // SENSOR_MS = 5 分  
鐘 float t, h; if (shtc3_read(t, h)) { // 讀取室內溫濕度 lastTemp = t; lastHum = h;  
lastStatus = "WiFi OK"; // 根據 WiFi 狀態更新 } lastSensor = now; } }
```

4.5 LCD 顯示佈局

17:45:32		← 時鐘（最上方，大字）
26.5°C	68.2%	← 室內溫濕度（SHTC3）
WiFi OK		← 狀態列

4.6 WiFi 設定參數

```
const char* WIFI_SSID = "IoT"; // WiFi 網路名稱 (SSID) const char* WIFI_PASS =  
"057851463"; // WiFi 密碼 const char* NTP_SERVER = "pool.ntp.org"; // NTP 網路時間伺服器  
const long GMT_OFFSET = 8 * 3600; // 時區：台灣 = UTC+8 (8小時×3600秒)
```

第五章：系統運作流程 — 從開機到顯示

5.1 完整開機流程（時間軸）

1 T+0 秒 — 系統重置

ESP32 內部重置完成，CPU 開始執行燒錄進去的韌體。

2 T+0.3 秒 — LCD 初始化

ST7305 RLCD 驅動 IC 啟動，LCD 螢幕開始接收指令，畫面為空白。

3 T+0.5 秒 — SHTC3 喚醒

I2C 匯流排啟動（GPIO13/14），韌體發送 wakeup 命令讓 SHTC3 離開睡眠模式，接著執行軟體重置。

第一次 `shtc3_read()` 嘗試讀取室內溫濕度。

4 T+1 秒 — WiFi 嘗試連線

ESP32 開始掃描 SSID 「IoT」 並嘗試連線。最多等待 20 次 $\times$ 0.5 秒 = 10 秒。若成功，`WiFi.status()` == WL_CONNECTED。

5 T+5-10 秒 — NTP 同步

WiFi 連線成功後，向 `pool.ntp.org` 發送 NTP 請求，取得網路時間（UTC），加上 GMT+8 時區偏移，轉換為台灣本地時間。時間寫入 PCF85063 RTC 晶片。

6 T+10 秒 — 進入正常顯示模式

- 時鐘：每秒從 RTC 讀取並更新顯示
- 溫濕度：每 5 分鐘從 SHTC3 讀取一次並更新顯示
- 狀態列：根據 WiFi/SHTC3 狀態即時更新

5.2 斷線處理機制

情境	發生的狀況	如何恢復
WiFi 瞬斷（幾秒內恢復）	狀態列短暫顯示「WiFi...」後自動恢復	自動重連，不需人工介入
WiFi 長時間斷線	狀態列顯示「WiFi OFF」，時鐘繼續從 RTC 運行（RTC 有備用電池）	WiFi 恢復後自動重連，NTP 會再次同步
SHTC3 讀取失敗	狀態列顯示「Sensor FAIL」，最後一次讀到的溫濕度值維持不變	每 5 分鐘自動重試，5 次連續失敗後仍持續重試
完全斷電（拔除 USB）	RTC 備用電池維持時間約 1 年，WiFi 設定保存在 Flash	重新供電後，WiFi 自動連線，RTC 時間已保存

5.3 電力消耗分析

預估耗電（典型使用情境）：

- WiFi 傳輸期間：約 80-100mA
- SHTC3 測量期間（每 5 分鐘一次，每次約 20ms）：約 0.8mA
- 靜態顯示（時鐘運行，WiFi 待機）：約 20-30mA
- RLCD 靜態顯示：幾乎不耗電（不需要背光）

結論：使用 18650 鋰電池供電時，預估可連續運行 3-5 天（取決於 WiFi 訊號強度與更新頻率）。

第六章：燒錄與更新 — 把韌體放進 ESP32

6.1 燒錄前置作業

⚠ 燒錄前置檢查清單：

- ✓ ESP32 已透過 USB-C 連接到 VM
- ✓ VM 已確認能偵測到 /dev/ttyACM0
- ✓ VM 有存取網際網路（下載 Arduino Core 用）
- ✓ USB 權限已設定（chmod 666 /dev/ttyACM0）

6.2 完整燒錄流程（Step by Step）

1 設定路徑並確認 Arduino CLI 可用

```
$ export PATH="$PATH:/home/jhe/.openclaw/workspace/bin" $ arduino-cli version
arduino-cli Version: 1.2.0
```

2 編譯韌體（不燒錄，確認編譯成功）

```
$ arduino-cli compile \ --fqbn
esp32:esp32:esp32s3:PSRAM=enabled,PartitionScheme=default \ --library
"/home/jhe/.openclaw/workspace/esp32-rlcd-
project/01_Arduino_Libraries/ST7305_U8g2" \ --library
"/home/jhe/.openclaw/workspace/esp32-rlcd-
project/01_Arduino_Libraries/SensorLib/src" \ --library
"/home/jhe/.openclaw/workspace/esp32-rlcd-project/01_Arduino_Libraries/lvgl9" \ -
--library "/home/jhe/.openclaw/workspace/esp32-rlcd-
project/01_Arduino_Libraries/U8g2" \ /home/jhe/.openclaw/workspace/esp32-rlcd-
project/02_Example/Arduino/11_Weather_Clock/11_Weather_Clock.ino
```

編譯成功後，產物會在 `/tmp/esp32_build/` 目錄下。其中最重要的是 `*.merged.bin`，這個檔案包含了完整的韌體（Bootloader + Partition Table + App）。

3 使用 esptool 燒錄到 ESP32

```
$ python3 -m esptool \ --chip esp32s3 \ --port /dev/ttyACM0 \ --baud 921600 \
write_flash \ 0x0 /tmp/esp32_build/11_Weather_Clock.ino.merged.bin
```

參數說明：

- `--chip esp32s3`：目標晶片型號
- `--port /dev/ttyACM0`：USB 序列埠（燒錄時不能被其他程式佔用）
- `--baud 921600`：燒錄速度（921600 bps，約 12 秒燒完 4MB）
- `0x0`：Flash 起始位址（從 0x0 開始寫入）

4 燒錄成功確認

```
Writing at 0x00000000 [100.0%] 4194304 bytes... Wrote 4194304 bytes (626986
```

```
compressed) at 0x00000000 in 12.5 seconds Verifying written data... Hash of data
verified. Hard resetting via RTS pin...
```

出現 `Hash of data verified.` 表示燒錄成功且資料完整！

5 燒錄後驗證

```
$ python3 -m esptool --chip esp32s3 --port /dev/ttyACM0 --baud 115200 read-mac
Connected to ESP32-S3 on /dev/ttyACM0: Chip type: ESP32-S3 (QFN56) (revision
v0.2) MAC: 3c:dc:75:fc:0d:b8
```

6.3 修改韌體時的快速燒錄技巧

修改 `11_Weather_Clock.ino` 後，只需要兩道指令：

```
$ # Step 1: 編譯 (輸出到 /tmp/esp32_build/) $ arduino-cli compile --fqbn
esp32:esp32:esp32s3:PSRAM=enabled \ "/home/jhe/.openclaw/workspace/esp32-rlcd-
project/02_Example/Arduino/11_Weather_Clock/11_Weather_Clock.ino" \ --output-dir
/tmp/esp32_build/ $ # Step 2: 燒錄 (直接使用 .merged.bin) $ python3 -m esptool --chip
esp32s3 --port /dev/ttyACM0 --baud 921600 \ write_flash 0x0
/tmp/esp32_build/11_Weather_Clock.ino.merged.bin
```

6.4 燒錄失敗的常見原因

錯誤訊息	原因	解決方案
「Connected...」一直等待	USB 權限不足或被其他程式佔用	<code>sudo chmod 666 /dev/ttyACM0</code> 或 kill 佔用程式
「Insufficient flash size」	韌體太大，超過 Flash 容量	使用 default Partition Scheme 而非 minimal
「Hash of data verified」但畫面不亮	SPI 接線問題或 LCD 初始化失敗	檢查 GPIO 11/12/40/41 硬體連接
SHTC3 一直讀不到	I2C 位址錯誤或未執行 wakeup	確認 I2C 位址為 0x70，wakeup 已在 <code>setup()</code> 中呼叫

第七章：伺服器端設定 — 天氣資料定時更新

雖然本專案 ESP32 只顯示室內 SHTC3 感測器的資料，但 ESP32 原本也有抓取網路天氣 JSON 的功能（可在程式碼中隨時切換）。以下說明 VM 端的排程服務。

7.1 系統服務架構

ESP32 室內溫濕度時鐘的 VM 端服務：

- 排程服務：使用 Linux `cron`（系統級，永久生效）
- 排程檔案：`/etc/cron.d/jhe-crons`
- 韌體更新後無需調整 VM 設定

7.2 目前的排程列表（共 20+1 個任務）

排程時間	執行動作	說明
每小時 :00	cron-status-update.sh	系統狀態頁面更新（溫度、發電量、磁碟空間）
每 10 分鐘	cron-stock-update.sh	台股/美股持股報價更新
每 30 分鐘	fetch-weather-to-r2.sh	天氣資料上傳至 R2 CDN
每天 08:00 / 12:00 / 18:00	wind-alert.sh	風速警示通知（Telegram）
每小時 :00	esp32_weather_update.sh	ESP32 天氣 JSON 更新（每小時）
每天 09:00	cron_us_dividend.py	美股除息行事曆
每天 20:00	update_dividend_data.py	台股除息資料更新
每天 15:00	gen_portfolio_data.py	持股總攬 JSON 產生
每週三、六 10:00	crypto_report.py	加密貨幣分析報告
每週日 09:00	us_tech_report.py	美股科技七巨頭報告
每 3 天 16:00	etl_memory_v3.py	記憶向量資料庫同步

7.3 將排程寫入系統（VM 重開機後保留）

```
$ # 將排程寫入 /etc/cron.d/（系統級，VM 重開機不會消失） $ sudo cp crontab.bak
/etc/cron.d/jhe-crons $ sudo chmod 644 /etc/cron.d/jhe-crons $ cat /etc/cron.d/jhe-
crons # 確認寫入成功
```

⚠ 注意：千萬不要把排程寫在使用者 crontab（`crontab -e`）裡，因為 VM 重開機後使用者 crontab 會歸零。使用 `/etc/cron.d/` 目錄下的檔案才能永久保存。

7.4 ESP32 天氣 JSON（可供日後切換回網路天氣用）

VM 端目前產生的天氣 JSON 檔案位置：

```
# VM 端產生的天氣 JSON（ESP32 預設 URL） https://pub-
ad498842971c4801a54fabd88ffa4a7f.r2.dev/tmp/weather.json
```

如日後要將 ESP32 改回顯示「室外天氣」（網路天氣），只需修改 `11_Weather_Clock.ino` 中的 `WEATHER_URL` 為上述網址，並啟用原來的 `fetchWeather()` 函式即可。

第八章：疑難排解 — 常見問題與解決方案

8.1 燒錄相關問題

Q1：燒錄時一直顯示「Connecting...」

原因：USB 權限不足、或有其他程式佔用序列埠。

解決：

```
$ sudo chmod 666 /dev/ttyACM0 $ lsof /dev/ttyACM0 # 查看是否被佔用
```

Q2：燒錄完成但 ESP32 沒反應

檢查清單：

- USB-C 供電是否足夠（建議 5V 2A 以上）
- GPIO 0（BOOT 按鈕）是否在燒錄時被按下
- 板子上的 BOOT 和 RST 按鈕，同時按 RST 再放開可以強制進入下載模式

8.2 SHTC3 感測問題

Q3：SHTC3 讀取一直失敗（顯示 Sensor FAIL）

步驟 1：確認 I2C 是否正常

```
$ python3 -c "import smbus b = smbus.SMBus(1) print('I2C devices:', [hex(x) for x in b.read_i2c_block_data(0x70, 0xefc8, 2)])"
```

步驟 2：確認 SHTC3 I2C 位址 — 正確位址為 `0x70`（不是 `0x38` 或其他）

步驟 3：確認 wakeup 是否執行 — `Wire.begin(13, 14)` 必須在第一次 `shtc3_read()` 之前執行

Q4：SHTC3 讀到的溫度值很奇怪（如 -40°C 或 200°C）

原因：I2C 資料解析錯誤，通常是 CRC 干擾或時序問題。

解決：增加 `delay(20)` 等待時間，或將 `Wire.setClock(400000)`（400kHz I2C）加入 `setup()`。

8.3 WiFi 相關問題

Q5：WiFi 一直連不上

檢查順序：

1. 確認 SSID 和密碼正確（注意大小寫）
2. 確認 WiFi 是 2.4GHz（ESP32-S3 不支援 5GHz WiFi）
3. 確認路由器沒有 MAC 位址過濾
4. 將 ESP32 靠近路由器測試，排除訊號強度問題

Q6：NTP 時間一直同步失敗

原因：WiFi 連線成功但無法連接外網（NTP），或防火牆封鎖 UDP 123 連接埠。

解決：在家用網路環境下通常不受限制，可嘗試更換 NTP 伺服器：

```
const char* NTP_SERVER = "time.google.com"; // Google NTP
const char* NTP_SERVER = "tick.stdtime.gov.tw"; // 台灣國家標準時間
```

8.4 顯示相關問題

Q7：LCD 螢幕不亮（完全黑的）

檢查重點：

- 確認 LCD 排線已正確插入（FPC 連接器）
- 確認 GPIO 11/12（SPI）、GPIO 5（DC）、GPIO 40（CS）、GPIO 41（RST）正確無誤
- RLCD 需要幾秒初始化時間，上電後等待 2-3 秒才會開始顯示
- 全反射 LCD 在強光下顯示效果最佳，暗處可能看不清楚（正常現象）

Q8：LCD 顯示，但數值不更新

原因：ESP32 在空迴圈中卡住，或 `displayAll()` 函式未被執行。

解決：確認 `yield()` 在 `loop()` 中有被呼叫（處理 WiFi 後台任務），並檢查 `DISPLAY_MS = 1000` 是否正確。

8.5 VM 環境問題

Q9：VM 重開機後 crontab 變空

根本原因：使用者 crontab（`crontab -e`）在 VM 重開機後不保留。

解決：使用系統級 cron 檔案：

```
$ sudo nano /etc/cron.d/jhe-crons # 在這裡寫入排程，永久生效
```

Q10：ESP32 USB 裝置無法在 VM 中偵測到

解決：在 VMware 選單中：

VM → 可移除裝置 → USB → 連接（名稱如 "USB Serial Device"）

連線後 VM 內就能看到 `/dev/ttyACM0` 了。

附錄

A. GPIO 腳位對照表

GPIO	功能	本專案用途
GPIO 11	SPI CLK	LCD SPI 時脈
GPIO 12	SPI MOSI	LCD 資料輸出
GPIO 13	I2C SDA	SHTC3 / RTC I2C 資料
GPIO 14	I2C SCL	SHTC3 / RTC I2C 時脈
GPIO 40	GPIO (SPI CS)	LCD 片選
GPIO 41	GPIO (SPI RST)	LCD 重置
GPIO 5	GPIO	LCD DC (資料/命令切換)
GPIO 0	BOOT 按鈕	燒錄時使用（保持按下可進入下載模式）

B. 重要檔案路徑

檔案	路徑
韌體主程式	/home/jhe/.openclaw/workspace/esp32-rlcd-project/02_Example/Arduino/11_Weather_Clock/11_Weather_Clock.ino
SHTC3 範例	/home/jhe/.openclaw/workspace/esp32-rlcd-project/02_Example/Arduino/05_I2C_SHTC3/05_I2C_SHTC3.ino
Arduino CLI	/home/jhe/.openclaw/workspace/bin/arduino-cli
ESP32 Core	/home/jhe/.arduino15/packages/esp32/
韌體燒錄產物	/tmp/esp32_build/11_Weather_Clock.ino.merged.bin
排程設定	/etc/cron.d/jhe-crons

C. 參考資源

- Waveshare ESP32-S3-RLCD-4.2 官方文檔：<https://docs.waveshare.net/ESP32-S3-RLCD-4.2.htm>
- Sensirion SHTC3 資料手冊：<https://sensirion.com/products/catalog/SHTC3/>
- Arduino CLI 官方文檔：<https://arduino.github.io/arduino-cli/>
- ESP32 Arduino Core (GitHub)：<https://github.com/espressif/arduino-esp32>

ESP32 室內溫濕度時鐘建置教學 | v1.0 | 2026-06-29
適用硬體：Waveshare ESP32-S3-RLCD-4.2 | 感測器：SHTC3